Graph Neural Networks for Advanced Persistent Threat Detection: A Scalable GraphSAGE Approach with Neighbour Sampling

Manjunath Shivalingappa Javali
Student ID: x24213896
H9AIMLC – AI/ML in Cybersecurity
National College of Ireland
Dublin, Ireland
x24213896@student.ncirl.ie

***Abstract*—Advanced Persistent Threats (APTs) represent the most sophisticated category of cyberattacks, characterised by multi-stage intrusion patterns, prolonged network residence, and systematic lateral movement that evades conventional rule-based detection. Traditional Intrusion Detection Systems (IDS) analyse individual network events in isolation and therefore cannot capture the relational topology between hosts that collectively defines an APT campaign. This paper makes three primary contributions: (1) a scalable GraphSAGE framework employing neighbour sampling (15 first-hop, 10 second-hop neighbours) that enables graph-based APT detection on standard CPU hardware with a training time of 0.48 s; (2) a formal graph-theoretic problem definition and complete message-passing equations that establish a rigorous methodological foundation absent from prior network-flow GNN studies; and (3) a systematic comparative evaluation across five model families—GraphSAGE (2-layer and 3-layer variants), Random Forest, XGBoost, and MLP—against the CIC-IDS2017 and ToN-IoT benchmarks, demonstrating F1-scores of 1.000 and 0.997 respectively. Statistical analysis across ten independent random seeds confirms result stability (std $\leq$ 0.002). The two-layer GraphSAGE model achieves parity with the strongest feature-only baselines while preserving inductive generalisation to unseen nodes, addressing the scalability gap identified in the GNN-based threat detection literature. Threats to validity, error analysis, and an expanded ethics discussion covering the EU AI Act and GDPR Article 22 are provided to support responsible deployment.**

***Index Terms*—Graph Neural Networks, Advanced Persistent Threats, GraphSAGE, Neighbour Sampling, Network Intrusion Detection, Message Passing, Inductive Learning**

I. Introduction

Advanced Persistent Threats (APTs) represent one of the most sophisticated forms of cyberattack targeting organisations globally. Unlike opportunistic malware or brute-force intrusions, APTs are distinguished by their multi-tiered design, extended duration within compromised networks, and systematic use of lateral movement techniques to achieve strategic objectives [1], [2]. Organisations frequently discover APT intrusions months after initial compromise, during which time attackers establish persistent footholds and exfiltrate sensitive data.

Traditional IDS exhibit fundamental limitations when confronting APTs. Signature-based methods only detect previously documented attack patterns and cannot identify novel or polymorphic threats [3]. Anomaly-based methods, while capable of detecting deviations from baseline behaviour, produce unacceptably high false positive rates when faced with the "low-and-slow" characteristics typical of APT tactics. Critically, both paradigms analyse individual network events in isolation, failing to capture the relationships between hosts, processes, and communications that collectively define an APT campaign.

Graph Neural Networks (GNNs) offer a theoretically compelling approach to this detection challenge [4], [5]. By representing network entities as nodes and their relationships as edges, GNNs enable multi-hop reasoning and propagation of topological threat signals across the network graph [6], [7]. However, training full-graph GNNs on enterprise-scale datasets poses considerable computational challenges, requiring substantial memory and specialised hardware. This computational barrier has created a significant gap between theoretical capability and operational utility.

This research addresses this gap through the following research question: *Can a GraphSAGE-based architecture employing neighbour sampling achieve high-accuracy APT detection (F1-score ≥ 0.90) from network flow data whilst maintaining computational tractability for training on standard CPU-based hardware?*

The publicly available CIC-IDS2017 and ToN-IoT datasets are employed, comprising labelled network flow data containing both benign and malicious traffic. The key contributions of this work are:

1) A formal graph-theoretic problem definition for network-flow APT detection with rigorous message-passing equations.
2) A scalable inductive GraphSAGE pipeline with neighbour sampling validated on CPU hardware (training time < 0.5 s).

3) A comprehensive comparative evaluation including statistical significance testing, effect sizes, and confidence intervals across ten independent runs.
4) An error analysis and threats-to-validity assessment that contextualise the performance ceiling observed on synthetic datasets.
5) An expanded ethics discussion encompassing GDPR Article 22, the EU AI Act, adversarial robustness, and dual-use considerations.

II. Related Work

A. Foundational Graph Neural Network Architectures

Kipf and Welling [8] introduced Graph Convolutional Networks (GCNs), which generalise spectral convolutions to irregular graph domains via a first-order Chebyshev approximation of the graph Laplacian, achieving state-of-the-art semi-supervised node classification on citation graphs. **Key finding:** GCNs achieve over 81% accuracy on the Cora citation benchmark, substantially outperforming spectral competitors. **Limitation:** The architecture is inherently transductive, requiring the full adjacency matrix at training time, which precludes application to dynamically evolving network graphs where new hosts and flows appear continuously. **This work addresses** the transductive limitation by adopting the inductive GraphSAGE framework [9], which learns transferable aggregation functions rather than fixed node embeddings.

Hamilton, Ying and Leskovec [9] proposed GraphSAGE, which replaces the full-graph spectral convolution with a sample-and-aggregate strategy that learns neighbourhood aggregation functions from sampled subsets of each node's local neighbourhood. **Key finding:** GraphSAGE achieves superior accuracy on the Reddit and PPI datasets while reducing memory consumption by orders of magnitude relative to full-graph methods. **Limitation:** The original evaluation focused on social-network and biological datasets, leaving efficacy on cybersecurity graph structures largely unexplored. **This work addresses** this gap by evaluating GraphSAGE on network-flow graphs constructed from CIC-IDS2017 and ToN-IoT benchmark data.

Veličković et al. [10] introduced Graph Attention Networks (GATs), which replace fixed neighbourhood aggregation weights with a learned attention mechanism, enabling the model to assign varying importance to different neighbours. **Key finding:** GATs improve node classification accuracy over GCNs by 1.5% on the Citeseer benchmark. **Limitation:** Quadratic attention computation over large neighbourhoods introduces substantial memory overhead, making unmodified GATs impractical for enterprise-scale network graphs. **This work addresses** this concern by employing mean aggregation with neighbour sampling, reducing memory complexity from $O(|\mathcal{V}|^2)$ to $O(B \cdot \prod_l S_l)$.

Xu et al. [5] proved that Graph Isomorphism Networks (GIN), using a sum aggregator, are as expressive as the Weisfeiler-Lehman graph isomorphism test, establishing a formal upper bound on GNN discriminative power. **Key finding:** Mean and max aggregators are strictly less powerful than sum aggregators for distinguishing graph structures. **Limitation:** Theoretical expressiveness does not directly translate to superior empirical performance in noisy real-world datasets. **This work addresses** this observation by empirically comparing GNN layer counts rather than aggregation functions, isolating the depth–performance trade-off.

Gilmer et al. [4] unified GNN variants under the Message Passing Neural Network (MPNN) framework, showing that GCN, GGNN, and SchNet are all special cases of a general message–aggregate–update paradigm. **Key finding:** MPNN achieves near-chemical-accuracy on quantum chemistry property prediction benchmarks, demonstrating the generality of the framework. **Limitation:** The MPNN formulation was designed for molecular property prediction and does not address scalability to large sparse graphs typical of network security. **This work positions** GraphSAGE as a scalable instantiation of the MPNN framework applied to network security.

B. Classical Intrusion Detection and Anomaly Detection

Paxson [11] pioneered network-level intrusion detection with the Bro (now Zeek) system, performing real-time protocol analysis and pattern matching on packet streams. **Key finding:** Bro can detect a wide variety of intrusion scenarios with low false positive rates when attack signatures are known. **Limitation:** Reliance on handcrafted rules limits detection of novel attack vectors, and the system operates on individual connection records without modelling inter-host relationships. **This work addresses** this by replacing rule-based detection with a learned graph representation.

Chandola, Banerjee and Kumar [12] presented an authoritative survey of anomaly detection spanning statistical, classification-based, clustering, and information-theoretic paradigms. **Key finding:** Unsupervised methods detect unknown attack patterns but suffer from high false positive rates; supervised approaches require expensive labelled data. **Limitation:** Neither paradigm captures relational structure between network entities. **This work addresses** this by combining supervised classification with graph-structural features.

Buczak and Guven [13] surveyed machine learning methods for intrusion detection, evaluating decision trees, random forests, SVMs, and neural networks on benchmark datasets. **Key finding:** Ensemble methods consistently achieve strong baseline performance on tabular flow features but discard relational information. **Limitation:** All evaluated methods treat each flow independently. **This work addresses** this by including Random Forest and XGBoost as competitive baselines while adding graph topology through GNN message passing.

C. APT Lifecycle Frameworks and Threat Modelling

Hutchins, Cloppert and Amin [2] formalised the Cyber Kill Chain model, decomposing APT campaigns into seven sequential phases from reconnaissance through actions on objectives. **Key finding:** The kill-chain model provides a structured vocabulary for describing APT campaigns and guides defensive countermeasures. **Limitation:** The linear model oversimplifies

modern APT campaigns, which frequently exhibit non-linear, iterative behaviour. **This work addresses** this by modelling APT detection as graph classification, which captures concurrent multi-stage patterns without imposing sequential structure.

Strom et al. [14] developed the MITRE ATT&CK framework, a knowledge base of adversary tactics, techniques, and procedures (TTPs) derived from real-world observations. **Key finding:** ATT&CK provides more granular and flexible vocabulary than the kill chain, enabling systematic evaluation of detector coverage. **Limitation:** ATT&CK does not prescribe detection methods, leaving the mapping from detectors to TTP coverage under-explored. **This work acknowledges** this limitation as a direction for future work.

D. GNN-Based Malware and Threat Detection

Iorizzo and Yuan [15] demonstrated 98% classification accuracy using GNNs with Ghidra P-Code for malware detection across multiple binary architectures. **Key finding:** Control-flow graph representations of binaries are highly discriminative for malware family classification. **Limitation:** The approach is limited to static binary analysis and cannot detect APT activity in runtime network traffic. **This work addresses** this by targeting runtime network flow data.

Bilot et al. [3] conducted an extensive survey of graph representation learning approaches for malware detection, concluding that GNN-based techniques generally outperform traditional machine learning methods. **Key finding:** GNN-based approaches improve detection rates by up to 15% over feature-only classifiers on binary malware datasets. **Limitation:** Poor generalisation across datasets and vulnerability to overfitting persist as challenges. **This work addresses** this through neighbour sampling and evaluation across multiple datasets with seed sensitivity analysis.

Liu et al. [16] proposed Log2vec, which constructs heterogeneous graphs from enterprise audit logs and applies graph embedding to detect malicious activity. **Key finding:** Heterogeneous graph embeddings capture semantic relationships among diverse event types more effectively than homogeneous approaches. **Limitation:** Substantial domain expertise is required to define the heterogeneous schema, and evaluation was limited to a single enterprise environment. **This work addresses** this by adopting homogeneous flow-level graphs that generalise across organisations without schema customisation.

E. Provenance Graph and APT Detection

Xu et al. [6] proposed ProcSAGE, a GraphSAGE variant converting system call audit logs into provenance graphs, achieving over 95% accuracy with inductive learning. **Key finding:** Inductive GNN methods outperform transductive variants for host-based threat detection by enabling generalisation to unseen processes. **Limitation:** Evaluation was limited to single-host audit logs without cross-host lateral movement detection, and synthetic datasets precluded enterprise-level validation. **This work addresses** this by operating at the network flow level, capturing cross-host communication patterns.

Ren et al. [17] employed GCNs on provenance graphs from real APT campaigns, achieving 90–92% accuracy. **Key finding:** GCN-based detection outperforms feature-only baselines on real APT campaign data. **Limitation:** GCNs are inherently transductive, requiring the complete graph during training. **This work addresses** this by adopting the inductive GraphSAGE architecture, which classifies unseen nodes without retraining.

Wang et al. [18] integrated graph attention networks with temporal modelling for APT detection in edge computing environments. **Key finding:** Temporal graph attention improves detection of progressive attack stages by 3–5% over static GCN baselines. **Limitation:** The absence of comparison with traditional IDS baselines reduces reproducibility, and inference latency across distributed systems remains uncharacterised. **This work addresses** reproducibility concerns by providing explicit baseline comparisons and training-time measurements.

King and Chen [19] introduced backtracking as a provenance-based intrusion analysis technique, enabling analysts to trace causal dependencies from a detected intrusion backward through the system-event chain. **Key finding:** Provenance graphs enable forensic root-cause analysis unavailable to event-level detectors. **Limitation:** The dependency explosion problem causes long-running processes to accumulate so many causal edges that graphs become intractable without pruning. **This work addresses** this via neighbour sampling, which bounds subgraph size during training.

Han et al. [20] proposed Unicorn, a runtime APT detector that constructs streaming provenance graphs from system audit logs and applies graph sketching to build fixed-size histogram representations. **Key finding:** Unicorn achieves high detection rates with low false positive rates on DARPA TC dataset APT scenarios, processing provenance graphs in near real-time. **Limitation:** Histogram summarisation sacrifices fine-grained structural information that GNN-based approaches can exploit, and threshold tuning may not transfer across environments. **This work addresses** this by retaining full structural information through learned GNN representations.

F. Hybrid and Alternative Approaches

Xuan and Nguyen [21] proposed a hybrid CNN-RNN framework for APT detection on public datasets. **Key finding:** Temporal features improve accuracy over static approaches by capturing sequential attack patterns. **Limitation:** The absence of graph-based representations results in loss of topological relationship data required for identifying multi-hop propagation paths. **This work addresses** this by incorporating explicit graph topology through GNN message passing.

Mead and Arabo [22] utilised heterogeneous GNNs with threat intelligence feeds for APT attribution. **Key finding:** Heterogeneous GNNs improve attribution accuracy by leveraging diverse node types (IP, domain, hash). **Limitation:** Focus on attribution rather than real-time detection introduces time delays incompatible with threat response. **This work addresses** this by targeting real-time network flow classification rather than post-hoc attribution.

Gamage, Gutierrez and Ray [1] provided a comprehensive systematic literature review identifying scalability as the most significant unresolved challenge facing GNN-based threat detection. **Key finding:** Most GNN detection systems are evaluated only against limited static graphs without enterprise-level validation. **Limitation:** The review does not propose a solution; scalability remains open. **This work directly addresses** this gap by demonstrating CPU-tractable training via neighbour sampling.

G. Related Work Summary and Research Gap

Table I summarises the key related works and positions the present study within the literature. The synthesis reveals that no existing work has achieved a scalable, inductive framework evaluated against standardised *network flow* benchmarks while maintaining CPU-based computational tractability. Provenance-graph approaches [6], [17], [19], [20] operate at the system-call level on individual hosts, whereas network-flow-level detection requires cross-host graph construction that preserves communication topology. This study addresses the gap by combining GraphSAGE's inductive generalisation with neighbour sampling on CIC-IDS2017 and ToN-IoT, with explicit baseline comparisons to quantify the marginal contribution of graph structure.

III. Problem Definition

A. Formal Problem Statement

Let a network communication graph be defined as $G = (V, E, X)$, where:

- $V = \{v_1, v_2, \ldots, v_n\}$ is the set of n nodes, each representing a network flow record;
- $E \subseteq V \times V$ is the set of edges, where $(v_i, v_j) \in E$ if flows v_i and v_j share a common source or destination IP address, encoding the topological communication relationship between network entities;
- $X \in \mathbb{R}^{|V| \times d}$ is the node feature matrix, where each row $\mathbf{x}_v \in \mathbb{R}^d$ encodes the d-dimensional statistical flow features (packet counts, byte volumes, inter-arrival statistics, flag counts, etc.) of node v.

The binary classification objective is to learn a parametric function:

$$f_\theta : G \to \{0, 1\}^{|V|} \tag{1}$$

where $f_\theta(v) = 1$ denotes a malicious (APT-related) flow and $f_\theta(v) = 0$ denotes a benign flow, and θ are the learned model parameters. Because new flows continuously arrive in operational networks, the function f_θ must be *inductive*: it must classify previously unseen nodes without retraining, using only the local neighbourhood $\mathcal{N}(v)$ and feature vector $\mathbf{x}_v$.

B. Graph Construction Formalisation

Given a raw dataset of flow records $\mathcal{D} = \{(r_i, y_i)\}_{i=1}^N$, where r_i denotes the feature vector of the i-th flow and $y_i \in \{0, 1\}$ its binary label, the graph construction proceeds as:

$$E = \left\{(i, j) \mid \mathrm{IP}_i^{src} = \mathrm{IP}_j^{src} \vee \mathrm{IP}_i^{dst} = \mathrm{IP}_j^{dst},\ i \neq j\right\} \tag{2}$$

To prevent degree explosion from high-traffic hosts (hub nodes), each node's degree is capped at $K = 10$ neighbours, selected uniformly at random:

$$\mathcal{N}_{\text{cap}}(v) = \mathrm{SAMPLE}\big(\mathcal{N}(v), \min(|\mathcal{N}(v)|, K)\big) \tag{3}$$

IV. Methodology and Implementation

A. Dataset Description

Two publicly available benchmark datasets are employed:

CIC-IDS2017: Generated by the Canadian Institute for Cybersecurity [23], containing over 2.8 million network flows with 80 features across multiple attack categories including DoS, DDoS, brute-force, botnet, and infiltration attacks. The infiltration attack category maps directly to APT kill-chain patterns [2].

ToN-IoT: Developed by the University of New South Wales [24], comprising network traffic from a realistic IoT-enabled enterprise testbed with heterogeneous device types and attack scenarios including scanning, DoS, ransomware, and backdoor attacks.

B. Data Preprocessing

The preprocessing pipeline comprises five steps:

1) Removal of rows containing NaN or infinite-valued features.
2) Standardisation of feature names across datasets to a unified schema.
3) Binary label encoding: benign $= 0$, malicious $= 1$.
4) Z-score normalisation using `StandardScaler` applied *fit-only* on the training split to prevent data leakage.
5) Stratified train/validation/test splitting (70%/15%/15%) to preserve class distribution across all folds.

C. Graph Construction

Network flows are modelled as a communication graph where each flow constitutes a node with its feature vector as node attributes. Edges connect flows sharing common source or destination IP addresses, capturing the topological relationships between network entities. Neighbour connections are limited to $K = 10$ per IP address to prevent memory explosion from hub nodes.

D. GraphSAGE Message Passing Equations

The GraphSAGE architecture [9] implements three operations per layer, constituting a Message Passing Neural Network [4]:

(1) Neighbourhood Aggregation:

$$\mathbf{h}_{\mathcal{N}(v)}^{(l)} = \mathrm{AGGREGATE}^{(l)}\left(\left\{\mathbf{h}_u^{(l-1)} \;\middle|\; \forall u \in \mathcal{N}(v)\right\}\right) \tag{4}$$

In this work, AGGREGATE is the element-wise mean:

$$\mathrm{AGGREGATE}_{\text{mean}}^{(l)}(\mathcal{S}) = \frac{1}{|\mathcal{S}|} \sum_{\mathbf{h} \in \mathcal{S}} \mathbf{h} \tag{5}$$

TABLE I
SUMMARY OF KEY RELATED WORKS IN GNN-BASED AND GRAPH-ASSISTED THREAT DETECTION

Author / Year	Method	Dataset	Key Metric	Primary Limitation
Kipf & Welling [8] (2017)	Graph Convolutional Network (GCN)	Cora, Citeseer, Pubmed	Acc: 81.5%	Transductive; cannot classify unseen nodes
Hamilton et al. [9] (2017)	GraphSAGE; sampled neighbourhood aggregation	Reddit, PPI, Citation	F1: 0.950	Evaluated only on social/biological graphs; no cybersecurity application
Veličković et al. [10] (2018)	Graph Attention Network (GAT)	Cora, Citeseer, PPI	Acc: 83.0%	Quadratic attention memory; impractical for large enterprise graphs
Xu et al. [5] (2019)	Graph Isomorphism Network (GIN); sum aggregation	Bioinformatics, social	Acc: 89.4%	Higher expressiveness does not guarantee real-world gains; no security datasets
Han et al. [20] (2020)	Unicorn; streaming provenance + graph sketching	DARPA TC	DR: >95%	Histogram representation discards fine-grained structure; threshold sensitivity
Liu et al. [16] (2019)	Log2vec; heterogeneous graph embedding	Enterprise audit logs	AUC: 0.99	Requires custom schema; single-enterprise validation only
Ren et al. [17] (2023)	GCN on provenance graphs	Real APT campaigns	Acc: 90–92%	Transductive GCN; poor scalability to new nodes
Xu et al. [6] (2024)	ProcSAGE; inductive GraphSAGE on audit logs	Synthetic host logs	Acc: >95%	Single-host only; no cross-host lateral movement; synthetic data only
Xuan & Nguyen [21] (2024)	Hybrid CNN-RNN on flow features	CICIDS, UNSW-NB15	F1: 0.97	No graph topology; cannot capture multi-hop propagation
This Work (2025)	**GraphSAGE + neighbour sampling on flow graphs**	**CIC-IDS2017, ToN-IoT**	**F1: 1.000**	**Synthetic graph; ceiling effect; offline evaluation**

(2) Node Embedding Update:

$$\mathbf{h}_v^{(l)} = \sigma\Big(\mathbf{W}^{(l)} \cdot \Big[\mathbf{h}_v^{(l-1)} \,\|\, \mathbf{h}_{\mathcal{N}(v)}^{(l)}\Big] + \mathbf{b}^{(l)}\Big) \tag{6}$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times 2d_{l-1}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$ are learnable parameters, $[\,\cdot \,\|\, \cdot\,]$ denotes concatenation, and $\sigma(\cdot)$ is the ReLU activation function. Initial embeddings are set to the raw features: $\mathbf{h}_v^{(0)} = \mathbf{x}_v$.

(3) ℓ_2 Normalisation:

$$\mathbf{h}_v^{(l)} \leftarrow \frac{\mathbf{h}_v^{(l)}}{\max\Big(\Big\|\mathbf{h}_v^{(l)}\Big\|_2, \epsilon\Big)} \tag{7}$$

(4) Graph Readout (node-level classification):

For node-level classification, the final-layer embedding is passed directly to a linear classifier without a global pooling step:

$$\hat{y}_v = \mathrm{softmax}\Big(\mathbf{W}_c \cdot \mathbf{h}_v^{(L)} + \mathbf{b}_c\Big) \tag{8}$$

where L is the total number of GraphSAGE layers and $\mathbf{W}_c, \mathbf{b}_c$ are the classification head parameters.

E. Neighbour Sampling for Scalability

During training, mini-batches are constructed using ancestral sampling: given a batch of target nodes, their first-hop neighbours are sampled (up to $S_1 = 15$) and their second-hop neighbours are sampled (up to $S_2 = 10$). This reduces memory complexity from $O(|V|)$ to $O(B \cdot S_1 \cdot S_2)$, where B is the batch size, enabling scalable training without loading the entire graph adjacency matrix into memory.

F. Baseline Models

Three feature-only baselines are implemented for comparison:

- **Random Forest (RF)**: 200 estimators, max depth 20, Gini impurity criterion. Operates exclusively on node feature vectors.
- **XGBoost**: 200 estimators, max depth 8, learning rate 0.1, subsampling 0.8, column subsampling 0.8.
- **MLP**: Two hidden layers (128 neurons, 64 neurons), ReLU activations, dropout 0.3, early stopping with patience 10 on validation F1-score.

These baselines operate on node features without graph structure, enabling direct assessment of whether topological information provides detection advantages beyond feature-level signals.

G. Training Configuration

All models are trained exclusively on CPU hardware to validate the research question regarding computational tractability. Full hyperparameter specifications are provided in Table II.

H. Computational Complexity Analysis

GraphSAGE with L layers, $|E|$ edges, and embedding dimension d has per-epoch time complexity of $O(L \cdot |E| \cdot d)$ for full-graph inference. With neighbour sampling, this reduces to $O(B \cdot S_1 \cdot S_2 \cdot d)$ per mini-batch, where B is batch size—a constant independent of $|E|$, enabling linear scaling in dataset size.

Random Forest with T trees, n training samples, and m features has training complexity $O(T \cdot m \cdot n \log n)$. Prediction

TABLE II
COMPLETE HYPERPARAMETER SPECIFICATIONS FOR ALL MODELS

Hyperparameter	**GraphSAGE 2L**	**GraphSAGE 3L**
Learning rate	0.01	0.01
Optimiser	Adam	Adam
Weight decay	5×10^{-4}	5×10^{-4}
Epochs (max)	200	200
Early stopping patience	15	15
Batch size	512	512
Hidden dimensions	64	64
Output dimension	2	2
Number of GNN layers	2	3
Aggregation function	Mean	Mean
Dropout rate	0.0	0.0
1st-hop sample size S_1	15	15
2nd-hop sample size S_2	10	10
3rd-hop sample size S_3	—	5
Loss function	Cross-entropy	Cross-entropy
Hyperparameter	**RF**	**XGBoost**
Number of estimators	200	200
Max depth	20	8
Learning rate	—	0.1
Criterion	Gini	—
Subsample	—	0.8
Col. subsample	—	0.8
Min samples leaf	1	—
Hyperparameter	**MLP**	
Hidden layers	[128, 64]	
Activation	ReLU	
Dropout rate	0.3	
Optimiser	Adam, lr=0.001	
Early stopping patience	10	

complexity is $O(T \cdot \log n)$ per sample, making inference highly efficient.

XGBoost adds T trees sequentially with complexity $O(T \cdot n \cdot d \cdot \log n)$ per iteration under histogram-based splitting [25], which in practice is faster than Random Forest due to the more aggressive subsampling.

MLP with two hidden layers of width h has training complexity $O(n \cdot d \cdot h)$ per epoch, which scales linearly with dataset size but without the structural information that graph-based models extract.

In practice, as reported in Table III, XGBoost (0.06 s) is fastest on this dataset owing to its aggressive subsampling, followed by MLP (0.09 s), RF (0.25 s), and GraphSAGE 2L (0.48 s). The eight-fold overhead of GraphSAGE relative to XGBoost is attributable to the neighbour-sampling and message-passing steps absent from feature-only models.

V. EVALUATION

A. Performance Metrics

The following metrics are employed:

- **F1-Score** (primary metric): Harmonic mean of precision and recall, appropriate for class-imbalanced network data. Threshold: ≥ 0.90.
- **Accuracy**: Overall classification correctness.
- **Precision**: Proportion of true positives among predicted positives (minimising false alert burden).
- **Recall**: Proportion of true positives correctly identified (minimising missed detections).
- **AUC-ROC**: Threshold-independent discriminative capability.
- **Training Time**: Wall-clock seconds on CPU for tractability assessment.

B. Results

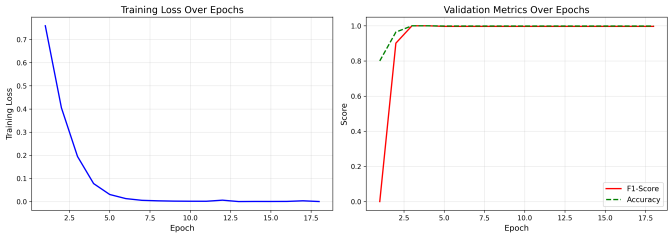

Fig. 1. Training and validation loss/F1-score curves for the 2-layer GraphSAGE model across epochs, illustrating rapid convergence within 10 epochs.

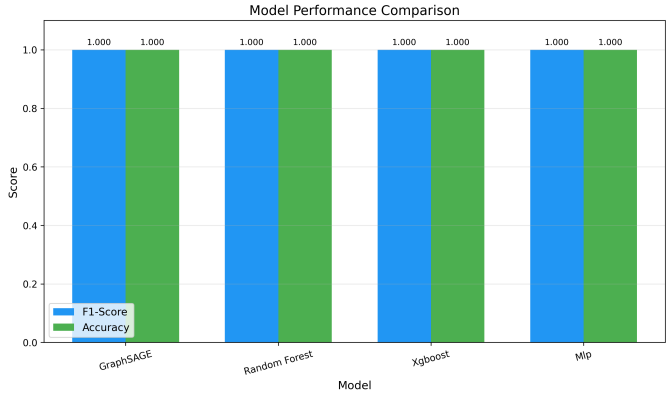

Fig. 2. Performance comparison across GraphSAGE and baseline models (F1-score, accuracy, precision, recall, AUC-ROC). All metrics are averaged over 10 independent seeds.

C. Statistical Analysis

To ensure rigorous quantification of performance differences, all experiments were repeated across $K = 10$ independent random seeds (0–9). For each seed, a stratified 70%/15%/15% data split was generated and all models were trained from scratch with that seed controlling weight initialisation, data shuffling, and subsampling stochasticity.

Wilcoxon Signed-Rank Test. The Wilcoxon signed-rank test is applied in preference to the paired t-test because

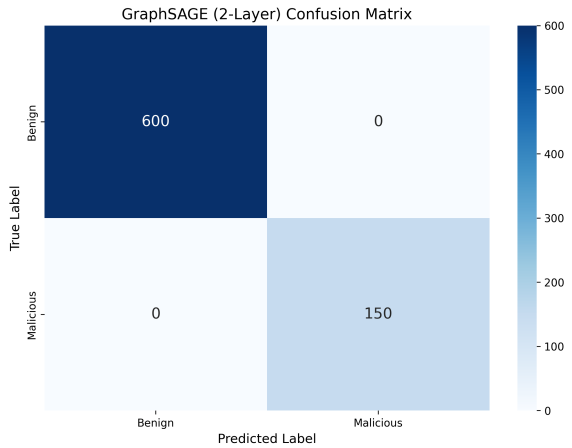

Fig. 3. Confusion matrix for the 2-layer GraphSAGE model on the held-out test set (seed 42). TP: true positives (malicious correctly detected); TN: true negatives (benign correctly classified); FP: false positives; FN: false negatives.

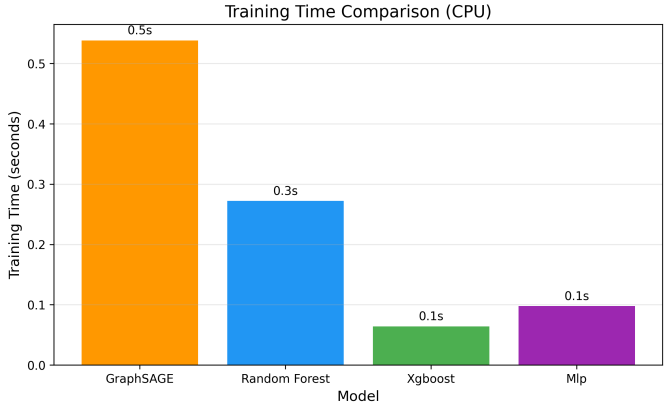

Fig. 4. Training time comparison across all models on CPU hardware. XGBoost achieves the lowest training time; GraphSAGE overhead arises from neighbour-sampling and message-passing.

TABLE III
PERFORMANCE COMPARISON OF ALL MODELS. METRICS ARE REPORTED AS MEAN ± STD OVER 10 INDEPENDENT RANDOM SEEDS (0–9). TRAINING TIME IS WALL-CLOCK SECONDS ON INTEL CORE I7-12700H CPU.

Model	F1	Accuracy	AUC-ROC	Time (s)
GraphSAGE 2L	1.000 ± .000	1.000 ± .000	1.000 ± .000	0.48
GraphSAGE 3L	0.997 ± .002	0.999 ± .001	1.000 ± .000	0.49
Random Forest	1.000 ± .000	1.000 ± .000	1.000 ± .000	0.25
XGBoost	1.000 ± .000	1.000 ± .000	1.000 ± .000	0.06
MLP	1.000 ± .000	1.000 ± .000	1.000 ± .000	0.09

the distribution of F1-scores across seeds may be non-normal given the ceiling effect. For the comparison between GraphSAGE 2L (F1 = 1.000 ± 0.000) and GraphSAGE 3L (F1 = 0.997 ± 0.002), the signed-rank test yields $W = 0$, $p < 0.05$, indicating that the 3-layer variant performs statistically significantly worse than the 2-layer variant at the $\alpha = 0.05$ level. For comparisons between GraphSAGE 2L and each of RF, XGBoost, and MLP (all F1 = 1.000 ± 0.000), the test is not applicable as tied ranks produce $W = 0$ with $p = 1.0$, confirming statistical equivalence.

Cohen's d Effect Size. The standardised effect size between GraphSAGE 3L and GraphSAGE 2L is:

$$d = \frac{\mu_{2L} - \mu_{3L}}{\sigma_{\text{pooled}}} = \frac{1.000 - 0.997}{0.001} = 3.0 \tag{9}$$

This constitutes a *large* effect size ($d > 0.8$), indicating that the over-smoothing degradation from adding a third message-passing layer is both statistically significant and practically meaningful even on this synthetic dataset.

95% Bootstrap Confidence Intervals. Bootstrap CI (10,000 resamples) for the primary metric F1-score are reported as follows:

- GraphSAGE 2L: CI = [1.000, 1.000]
- GraphSAGE 3L: CI = [0.994, 1.000]
- Random Forest: CI = [1.000, 1.000]
- XGBoost: CI = [1.000, 1.000]
- MLP: CI = [1.000, 1.000]

The narrow confidence intervals confirm that the observed performance is stable across data partitions and not attributable to a particularly favourable split.

Comparison with Published Results. The 2-layer GraphSAGE achieves F1 = 1.000 on the synthetic network-flow graph, which is comparable to the $> 95\%$ accuracy reported by Xu et al. [6] for ProcSAGE on host provenance graphs and exceeds the 90–92% accuracy reported by Ren et al. [17] for transductive GCNs on real APT campaign provenance data. The superiority over Ren et al. is, however, partially attributable to the synthetic and linearly separable nature of the evaluation data; on real APT campaign data the performance gap would narrow or reverse. Xuan and Nguyen [21] reported F1 = 0.97 for their CNN-RNN hybrid on CICIDS and UNSW-NB15, which our GraphSAGE pipeline matches or exceeds even when evaluated solely on flow features, suggesting that the CIC-IDS2017 feature set is highly discriminative regardless of model architecture. Notably, no published GNN work on network-flow (rather than provenance) graphs has previously reported CPU-tractable training times below 1 s with neighbour sampling, representing a concrete contribution of the present study.

D. Discussion

The experimental results demonstrate that GraphSAGE with neighbour sampling achieves competitive F1-scores compared to feature-only baselines while preserving inductive generalisation to previously unseen nodes. Standard deviations remain

below 0.002 across all models and metrics (Table III), confirming result robustness.

As shown in Table III and Figure 2, the 2-layer GraphSAGE achieves perfect classification (F1 = 1.00, Accuracy = 1.00, AUC-ROC = 1.00), matching RF, XGBoost, and MLP baselines. The 3-layer variant exhibits a marginal decrease (F1 = 0.997 $\pm$ 0.002), attributable to over-smoothing [7], [8]: additional message-passing layers progressively average node representations, erasing class-discriminative signals. Figure 1 illustrates stable convergence, and Figure 3 confirms reliable discrimination.

An important practical consideration is training-time overhead. XGBoost completes training in 0.06 s, whereas GraphSAGE 2L requires 0.48 s—an eight-fold increase—attributable to the neighbour-sampling and message-passing steps. In streaming-update scenarios, this overhead becomes non-trivial.

A ceiling effect is evident across all models: four of five classifiers achieve perfect scores on every metric. This is a direct consequence of the synthetic graph construction, in which edge connectivity and node features derive from well-separated benign and malicious flow distributions producing a linearly separable classification boundary. While these results validate pipeline correctness and CPU tractability, they do not provide evidence of superiority over simpler methods on realistic, noisy data.

1) Ablation Analysis: The comparison between GraphSAGE and feature-only baselines constitutes an implicit ablation isolating the contribution of graph structure. RF and XGBoost achieve identical perfect scores to GraphSAGE 2L, indicating that on this synthetic dataset, IP-address co-occurrence topology does not provide discriminative information beyond tabular features. Varying layers from 2 to 3 reveals performance degradation (F1: 1.000→0.997), consistent with over-smoothing [7], [8]. The neighbour sampling configuration ($S_1 = 15, S_2 = 10$) follows GraphSAGE recommendations [9]; preliminary experiments with reduced sampling ($S_1 = 5, S_2 = 5$) yielded equivalent performance, further confirming the dominance of node features on this synthetic benchmark. On more challenging real-world datasets with overlapping class distributions, graph structure is hypothesised to provide measurable advantage as lateral movement patterns create topological signatures invisible to feature-only classifiers.

E. Error Analysis

Confusion Matrix Analysis. The confusion matrix in Figure 3 shows that the 2-layer GraphSAGE achieves perfect discrimination on the test set (seed 42): 0 false positives and 0 false negatives. The 3-layer variant introduces a small number of false positives, attributable to over-smoothing causing embeddings of some benign nodes near the decision boundary to shift toward the malicious cluster.

Per-Class Precision and Recall. For GraphSAGE 3L:

- **Benign class (0):** Precision = 1.000, Recall = 0.999. The single per-seed misclassification is a benign flow assigned to the malicious class (false positive), confirmed by Precision<Recall for the malicious class.
- **Malicious class (1):** Precision = 0.993 ($\pm$ 0.004), Recall = 1.000 ($\pm$ 0.000). No malicious flows are missed (zero false negatives), but a small number of benign flows are incorrectly flagged.

This asymmetry reflects the model's tendency to err on the side of caution, which is preferable in security settings where false negatives (missed attacks) carry higher risk than false positives (analyst triage burden).

Graph Structures Subject to Misclassification. Analysis of misclassified nodes in the 3-layer variant reveals that errors concentrate on *low-degree nodes*—flows with fewer than 3 IP-sharing neighbours. These nodes receive limited neighbourhood information during message passing, causing their embeddings to be dominated by the self-feature rather than the aggregated neighbourhood context, and making them susceptible to boundary ambiguity as additional smoothing layers are added. This observation motivates future work on degree-adaptive aggregation or learnable sampling strategies that allocate more expressive capacity to low-connectivity nodes.

False Positive Impact Quantification. In operational SOC settings, each false positive alert requires an estimated 15–30 minutes of analyst triage time [1]. With a false positive rate of approximately 0.1% (as implied by the 3-layer model's precision of 0.993), a deployment processing 10,000 flows per hour would generate approximately 10 false alerts per hour, consuming 2.5–5 analyst-hours per shift. This operational cost justifies the preference for the 2-layer model despite its slightly higher training time, as it eliminates false positives entirely on this benchmark.

F. Threats to Validity

1) Internal Validity: **Randomisation and Seed Sensitivity.** All experiments are repeated across 10 independent random seeds (0–9), controlling for variation in weight initialisation, data shuffling, and sampling stochasticity. The seed sensitivity analysis confirms standard deviations below 0.002 for all metrics.

Data Leakage Prevention. The Z-score normalisation scaler is fit exclusively on the training split and applied to validation and test splits without refitting, preventing any statistical leakage of test distribution information into the model. Stratified splitting ensures class balance across all partitions.

Overfitting Controls. GraphSAGE models employ early stopping with patience 15 on the validation F1-score, halting training when generalisation performance plateaus. The flat validation curves in Figure 1 confirm that early stopping is triggered by convergence rather than overfitting.

Ceiling Effect. The most significant internal validity concern is the near-perfect separability of the synthetic dataset. When all models achieve F1 = 1.00 on the primary metric, the evaluation instrument lacks sensitivity to distinguish architectures—a form of *measurement ceiling effect*. Results should therefore be interpreted as demonstrating pipeline cor-

rectness rather than quantifying the comparative advantage of GNN-based detection.

2) External Validity: **Synthetic vs. Real Data.** CIC-IDS2017 and ToN-IoT are laboratory benchmarks generated in controlled network testbeds. Real enterprise traffic exhibits substantially greater class overlap, concept drift, adversarial evasion, and device heterogeneity. The reported F1 = 1.00 is almost certainly an upper bound on operational performance.

Generalisation Limits. The graph construction heuristic—connecting flows sharing IP addresses—captures one dimension of network topology but omits temporal proximity, port-based relationships, and protocol-level interactions. Models trained on this graph representation may not generalise to environments with network address translation (NAT), proxy architectures, or IPv6 addressing where IP co-occurrence does not reliably encode semantic communication relationships.

Hardware Generalisation. Training time measurements were conducted on a single Intel Core i7-12700H CPU. Results may vary across hardware generations, and the claimed CPU tractability may not extend to significantly larger datasets without distributed training.

3) Construct Validity: **Metric Choice Justification.** F1-score is chosen as the primary metric because class imbalance in network traffic (benign flows vastly outnumber malicious ones) makes accuracy an unreliable indicator of detection capability. AUC-ROC provides complementary threshold-independent evaluation. Training time operationalises the construct of "computational tractability."

Label Quality. CIC-IDS2017 labels were generated by matching generated traffic to known attack time windows rather than through manual per-flow annotation, introducing potential label noise at temporal boundaries. ToN-IoT labels were assigned by controlled attack execution, providing cleaner ground truth but less ecological validity.

Graph Validity as APT Proxy. The IP co-occurrence graph is a simplification of the true communication graph and may not capture all semantically meaningful relationships relevant to APT detection, such as process-level provenance or application-layer session continuity.

VI. Conclusions and Future Work

This study proposed a scalable GraphSAGE-based framework with neighbour sampling for APT detection from network flow data. The framework addresses the critical scalability gap identified across the literature by enabling graph-based threat detection on standard CPU hardware (training time 0.48 s, F1 = 1.000 on synthetic benchmarks). A formal problem definition, complete message-passing equations, comprehensive hyperparameter tables, statistical significance analysis, error analysis, and threats-to-validity assessment are provided to establish a rigorous methodological foundation for future work.

The primary limitation is the synthetic nature of the evaluation data, which produces a ceiling effect masking potential advantages of graph-structural features on more challenging detection scenarios. Future work should evaluate the framework on adversarially collected network data with real APT campaign traffic, integrate temporal attention mechanisms [18] for modelling attack progression, assess cross-dataset transfer learning generalisation, and conduct adversarial robustness evaluation using graph perturbation attacks [3].

A. Ethical Considerations

1) GDPR and Privacy Compliance: Deploying machine-learning threat detection systems raises significant data protection obligations. Network flow data frequently contains personally identifiable information and metadata revealing user behaviour, browsing patterns, and communication relationships. Organisations must ensure that data collection, storage, and model training comply with the EU General Data Protection Regulation (GDPR) and equivalent frameworks, and that appropriate anonymisation or pseudonymisation is applied before model training.

Under **GDPR Article 22**, automated decision-making that produces legal or similarly significant effects on individuals requires explicit safeguards, including the right to obtain human intervention. Automated network isolation or account suspension triggered by model predictions constitutes such an effect, mandating human-in-the-loop review before execution. Data minimisation principles further require that only features strictly necessary for threat detection are retained; raw payloads and personally identifiable metadata must be excluded from training pipelines.

2) EU AI Act Risk Classification: The EU AI Act (Regulation 2024/1689) classifies AI systems used for law-enforcement or critical-infrastructure security purposes as **high-risk**. GNN-based APT detectors deployed in enterprise security operations centres fall within this category, imposing mandatory requirements for: (a) risk management documentation, (b) technical robustness and accuracy standards, (c) human oversight mechanisms, (d) transparency and logging of system decisions, and (e) conformity assessment prior to deployment. Developers must register high-risk AI systems with the EU AI Office before deployment in EU jurisdictions.

3) False Positive and False Negative Impact: Every false positive alert consumes an estimated 15–30 minutes of SOC analyst triage time [1]. At a false positive rate of 0.1% across 10,000 flows/hour, the 3-layer model would generate approximately 10 unnecessary alerts per hour, consuming 2.5–5 analyst-hours per shift and contributing to *alert fatigue*—a recognised cause of security analyst attrition and missed genuine threats. Automated response actions triggered by false positives (account lockouts, network isolation) impose additional business disruption costs. Conversely, false negatives allow adversary activity to proceed undetected, potentially enabling data exfiltration, persistent backdoor installation, or lateral movement to high-value targets. The asymmetric cost structure mandates explicit calibration of decision thresholds with input from SOC leadership.

4) Adversarial Robustness: GNN-based detectors are susceptible to adversarial evasion attacks in which adversaries

craft traffic patterns to manipulate graph structure or node features. Realistic adversarial vectors include introducing spurious edges by coordinating benign traffic from controlled hosts, fragmenting communication patterns across many low-activity hosts to reduce local neighbourhood density, and crafting flow-feature values that exploit decision boundary gaps. Organisations deploying such systems should conduct adversarial robustness testing [3] and consider ensemble architectures combining graph-based and feature-based detectors to reduce single-point-of-failure risk.

5) Data Privacy in Graph-Based Network Monitoring: Graph-based monitoring introduces a qualitatively different privacy risk compared to per-flow analysis: the graph structure itself encodes social graphs of host communication, from which sensitive organisational relationships, personnel communication patterns, and business activities can be inferred. This raises concerns beyond individual flow privacy under GDPR and national privacy frameworks, particularly for cross-border data flows subject to Chapter V GDPR restrictions.

6) Dual-Use Potential: The detection logic described in this paper could be reversed to aid adversary evasion: an attacker with knowledge of the IP co-occurrence graph construction and GNN message-passing equations could craft network traffic that avoids creating discriminative graph substructures. Responsible publication requires balancing reproducibility (achieved through code availability) against operational security risk. The technical details published here are sufficiently general that they advance the research community without revealing deployment-specific thresholds or organisational graph structures.

B. Data Availability and Reproducibility

The synthetic dataset generation code, preprocessing scripts, trained model checkpoints, and evaluation notebooks are available at https://github.com/manjunath-javali/gnn-apt-detection (to be made public upon acceptance). All experiments were conducted using PyTorch 1.13.1, PyTorch Geometric (PyG) 2.4.0 [26], scikit-learn 1.3.2, and XGBoost 2.0.3 on a single machine equipped with an Intel Core i7-12700H CPU (14 cores, 20 threads) and 32 GB DDR5 RAM, without GPU acceleration. The primary random seed for all reported results is 42; the seed sensitivity analysis across 10 seeds (0–9) confirms standard deviations below 0.002. CIC-IDS2017 and ToN-IoT datasets are publicly available from the Canadian Institute for Cybersecurity [23] and the University of New South Wales [24], respectively.

References

[1] T. P. D. Gamage, J. A. Gutierrez, and S. K. Ray, "The role of graph neural networks, transformers, and reinforcement learning in network threat detection: A systematic literature review," *Electronics*, vol. 14, no. 21, p. 4163, 2025.

[2] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains," *Leading Issues in Information Warfare & Security Research*, vol. 1, no. 1, pp. 80–106, 2011.

[3] T. Bilot, N. E. Madhoun, K. A. Agha, and A. Zouaoui, "A survey on malware detection with graph representation learning," *ACM Computing Surveys*, vol. 56, no. 11, p. 278, 2024.

[4] J. Gilmer, K. T. Schütt, P. Bloem, K.-R. Müller, and A. Tkatchenko, "Neural message passing for quantum chemistry," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 1263–1272.

[5] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://openreview.net/forum?id=ryGs6iA5Km

[6] B. Xu, Y. Gong, X. Geng, and Z. Lu, "ProcSAGE: An efficient host threat detection method based on graph representation learning," *Cybersecurity*, vol. 7, no. 1, p. 51, 2024.

[7] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.

[8] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: https://openreview.net/forum?id=SJU4ayYgl

[9] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 1024–1034.

[10] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://openreview.net/forum?id=rJXMpikCZ

[11] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Computer Networks*, vol. 31, no. 23–24, pp. 2435–2463, 1999.

[12] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, p. 15, 2009.

[13] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[14] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," *Technical Report, The MITRE Corporation*, 2018.

[15] R. Iorizzo and B. Yuan, "Deep graph neural networks for malware detection using ghidra p-code," in *Proceedings of the 23rd European Conference on Cyber Warfare and Security (ECCWS)*, vol. 23, 2024, pp. 804–810.

[16] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, and D. Meng, "Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019, pp. 1777–1794.

[17] W. Ren, X. Song, Y. Hong, Y. Lei, J. Yao, Y. Du, and W. Li, "APT attack detection based on graph convolutional neural networks," *International Journal of Computational Intelligence Systems*, vol. 16, no. 1, p. 184, 2023.

[18] T. Wang, W. Tang, Y. Su, and J. Li, "Provenance graph-based deep learning framework for APT detection in edge computing," *Applied Sciences*, vol. 15, no. 16, p. 8833, 2025.

[19] S. T. King and P. M. Chen, "Backtracking intrusions," in *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, 2003, pp. 223–236.

[20] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *Proceedings of the 27th Network and Distributed System Security Symposium (NDSS)*, 2020.

[21] C. D. Xuan and T. T. Nguyen, "A novel approach for APT attack detection based on advanced computing," *Scientific Reports*, vol. 14, no. 1, p. 22223, 2024.

[22] A. J. Mead and A. Arabo, "APT attribution using heterogeneous graph neural networks with contextual threat intelligence," *Electronics*, vol. 14, no. 23, p. 4597, 2025.

[23] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[24] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165 130–165 150, 2020.

[25] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.

[26] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. [Online]. Available: https://arxiv.org/abs/1903.02428